SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1702**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____S_Hariram(1965260018)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Analytical Problem Solving

1. Water Jug Problem

Problem Statement

You are given two jugs: a 4-gallon jug and a 3-gallon jug. Neither jug has any measuring markings. There is a pump to fill the jugs with water. The objective is to get exactly 2 gallons of water into the 4-gallon jug.

Assumptions

- A jug can be filled from the pump.
- Water can be poured out of a jug onto the ground.
- Water can be poured from one jug into another.
- No other measuring devices are available.

Solution

State Representation

The state is represented as `(x, y)`, where:

- `x` = amount of water in the 4-gallon jug, where $x \in \{0, 1, 2, 3, 4\}$
- `y` = amount of water in the 3-gallon jug, where $y \in \{0, 1, 2, 3\}$

Initial State

`(0, 0)`

Both jugs are empty.

Goal State

`(2, y)`

where y can be 0, 1, 2, or 3. The main requirement is that the 4-gallon jug contains exactly 2 gallons.

Production Rules / Operators

1. **Fill the 4-gallon jug:**
   $(x, y) \rightarrow (4, y)$
2. **Fill the 3-gallon jug:**
   $(x, y) \rightarrow (x, 3)$
3. **Empty the 4-gallon jug:**
   $(x, y) \rightarrow (0, y)$
4. **Empty the 3-gallon jug:**
   $(x, y) \rightarrow (x, 0)$
5. **Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full.**
6. **Pour water from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full.**
7. **Pour all water from the 3-gallon jug into the 4-gallon jug when there is enough space.**
8. **Pour all water from the 4-gallon jug into the 3-gallon jug when there is enough space.**

Step-by-Step Procedure

Step	Action	4-Gallon Jug (x)	3-Gallon Jug (y)
0	Initial State	0	0
1	Fill the 3-gallon jug	0	3
2	Pour all water from 3-gallon jug into 4-gallon jug	3	0
3	Fill the 3-gallon jug again	3	3
4	Pour from 3-gallon jug into 4-gallon jug until it is full	4	2
5	Empty the 4-gallon jug onto the ground	0	2

6	Pour the 2 gallons from the 3-gallon jug into the 4-gallon jug	2	0

Result

The final state is:

`(2, 0)`

Therefore, **exactly 2 gallons of water are obtained in the 4-gallon jug.**

2. Mars Rover Agent Analysis & Exploration

Find out how Mars Rover analyses and explores samples on Mars by answering the following:

i. What are the percepts for this agent?

- **Visual/Optical Imagery:** High-resolution stereoscopic navigation cameras (Navcams), hazard avoidance cameras (Hazcams), mast cameras (Mastcam), microscopic/hand-lens imager (MAHLI).
- **Spectrometric & Analytical Data:** Laser-induced breakdown spectroscopy (LIBS / ChemCam / SuperCam), Alpha Particle X-ray Spectrometer (APXS), Raman spectroscopy, X-ray diffraction (CheMin), gas chromatography-mass spectrometry.
- **Physical/Proprioceptive Percepts:** Wheel odometry, wheel slip detection, tilt/pitch/roll sensors, IMU (Inertial Measurement Unit), motor torque sensors, temperature sensors.
- **Atmospheric/Environmental Percepts:** Wind speed/direction, atmospheric pressure, relative humidity, UV radiation, ambient surface temperature.
- **Telemetry & Ground Communication:** Signals, command sequences, and scheduling received from Earth (NASA/ESA ground control).

ii. Characterize the operating environment

Using Russell & Norvig's environment properties:

- **Partially Observable:** The rover cannot observe the entire planetary surface at once; subsurface structures and occluded terrain behind rocks/dunes are hidden.
- **Stochastic:** Slip on regolith, dust storms, mechanical wear, unpredictable terrain, and sudden sensor drift introduce randomness.
- **Sequential:** Actions taken now (e.g., driving toward a ridge or expending battery power) directly affect future choices and available energy.
- **Dynamic:** Environmental factors like dust accumulation on solar panels, shifting shadows, changing winds, and varying temperature gradients change independently of the rover's actions.
- **Continuous:** Space, time, speeds, orientation angles, battery levels, and sensor voltages operate across continuous ranges.
- **Single-Agent / Multi-Agent:** Primarily single-agent operating locally on the Martian surface (or loosely multi-agent when coordinating with orbiters/helicopters like Ingenuity).

iii. What are the actions the agent can take?

- **Locomotion & Navigation:** Move forward/backward, steer, turn in place, vary drive speed, engage obstacle avoidance maneuvers.
- **Sample Manipulation & Collection:** Deploy robotic arm, drill into rock surface, scoop regolith, abrasion/brushing of dust, cache rock cores into sealed sample tubes.
- **In-Situ Scientific Analysis:** Fire spectroscopic lasers, expose sample to X-rays, capture microscopic images, intake powder into internal analytical chemistry chambers.
- **Power & Thermal Management:** Orient solar panels, manage radioisotope thermoelectric generator (RTG) power distribution, toggle internal heating elements.
- **Communication:** Point high-gain and low-gain antennas to communicate directly with Earth (Deep Space Network) or relay data through Mars orbiters.

iv. How can one evaluate the performance of the agent? (Performance Measure)

- **Scientific Value:** Number, diversity, and geological significance of collected/analyzed rock and soil samples; scientific discoveries made.
- **Mission Safety & Survival:** Avoidance of unrecoverable hazard states (e.g., getting trapped in sand dunes, rollover, wheel damage, severe thermal damage).
- **Resource Efficiency:** Energy conservation, optimal battery/power utilization, and wear minimization.
- **Autonomy & Efficiency:** Distance traversed per sol, ratio of autonomous problem-solving vs. ground intervention requests, mission timeline adherence.
- **Data Fidelity:** Quality, resolution, and volume of scientific and imaging data successfully transmitted.

v. What sort of agent architecture do you think is most suitable for this agent? Why?

- **Most Suitable Architecture: Hybrid (Hierarchical) Agent Architecture** combining a **Goal-Based / Utility-Based Agent** with **Reactive Subsumption / Reflex layers**.
- **Justification:**
    - *Extremes of Latency:* Communication round-trip delay between Earth and Mars ranges from ~6 to 44 minutes. The rover cannot rely on real-time commands from Earth.
    - *Real-Time Safety (Reactive Layer):* For immediate hazards (e.g., sudden steep drop-offs, excessive wheel slip, collision risks), low-level reflex modules must trigger immediate stops without waiting for higher-level planning.
    - *Deliberative Planning (Goal-Based / Utility-Based Layer):* Path planning, target selection, resource scheduling (battery, memory, thermal), and scientific sample sequencing require deliberative reasoning over utility and goals.

- Therefore, a layered hybrid architecture ensures low-level safety reflexes run deterministically while high-level autonomy plans and optimizes mission goals.

3. 8-Queens Problem Formulation & Search Strategies

Problem Description

The 8-Queens problem requires placing 8 queens on an 8 × 8 chessboard such that no two queens attack each other. Therefore, no two queens can share the same row, column, or diagonal.

Problem Formulation Components

1. State Space Formulation

The state can be represented as an 8-tuple:

$S = (q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8)$

where $q_i \in \{1, 2, ..., 8\}$ represents the row position of the queen in column i.

This representation automatically ensures that no two queens occupy the same column.

The size of the search space is:

$8^8 = 16{,}777{,}216$ states

If the formulation is restricted so that every queen is placed in a different row, the search space is:

$8! = 40{,}320$ states

2. Initial State

In the incremental formulation, the initial state is an empty chessboard with zero queens placed.

In the complete-state formulation, all 8 queens are already placed on the board, usually in a random or default configuration.

3. Actions / Successor Function

In the incremental formulation:

- Place a queen in the next empty column.
- Try each row from 1 to 8.
- A queen is placed only if it does not conflict with previously placed queens.

In the complete-state formulation:

- Select any queen.
- Move it to another row in the same column.
- The move produces a new board configuration.

4. Goal Test

The goal is reached when all 8 queens are placed and no two queens attack each other.

For any two different queens i and j:

$q_i \neq q_j$

and

$|q_i - q_j| \neq |i - j|$

Therefore, the queens must not share the same row or diagonal.

Since each queen is already assigned to a different column, column conflicts are automatically avoided.

5. Path Cost

The path cost can be considered zero or a constant cost for each placement.

The main objective is to find a valid arrangement of 8 queens, so the cost of the path is not important.

Search Strategies

1. Backtracking Search (DFS with Constraint Checking)

Backtracking search places queens incrementally, usually one column at a time.

Steps:

1. Start with an empty chessboard.
2. Place a queen in the first column.
3. Move to the next column.
4. Try each possible row.
5. If the position is safe, place the queen.
6. If no safe position exists, go back to the previous column and change the queen's position.
7. Continue until all 8 queens are placed successfully.

Backtracking is efficient because it immediately prunes branches whenever a newly placed queen conflicts with an existing queen.

It can be improved using techniques such as Forward Checking and the Minimum Remaining Values (MRV) heuristic.

2. Local Search – Min-Conflicts / Hill Climbing

Local search starts with all 8 queens already placed on the chessboard, possibly in an incorrect configuration.

A common heuristic is:

h(n) = Number of pairs of attacking queens

The algorithm selects a queen that is involved in a conflict and moves it to another row in the same column.

The position that produces the lowest number of conflicts is selected.

This process continues until:

h(n) = 0

When h(n) = 0, no queens attack each other and the solution is found.

Conclusion

The 8-Queens problem can be formulated as a constraint satisfaction and search problem. Backtracking search systematically explores possible arrangements

while pruning invalid configurations. Local search methods such as Min-Conflicts can quickly find a valid solution by reducing the number of conflicts at each step.

4. OLA Cab Booking Problem-Solving Agent & Pseudocode

i. Identification of Problem-Solving Agent Type

- **Type of Agent: Utility-Based Problem-Solving Agent** (operating in a Goal-Based Framework).
- **Reasoning:**
    - A simple goal-based agent only checks if the cab reaches the destination.
    - In OLA cab booking, multiple cab types (mini, micro, sedan, shared, prime) can fulfill the goal, but they differ significantly across multiple competing attributes: **comfort level**, **fare (price)**, **estimated time of arrival (ETA)**, **travel duration**, and **capacity/sharing preference**.
    - A utility function $U(\text{option})$ quantifies the user's tradeoff preferences (e.g., balancing budget against comfort and speed) to select the optimal ride option.

ii. Pseudocode for OLA Cab Booking Agent

```
class OLACabBookingAgent:

    def __init__(self, user_profile, comfort_weight, cost_weight, time_weight):

        self.user_profile = user_profile

        self.w_comfort = comfort_weight

        self.w_cost = cost_weight

        self.w_time = time_weight
```

```
def utility(self, cab_option):
    # Utility function balancing comfort, estimated fare, and wait time
    u = (self.w_comfort * cab_option.comfort_rating) - \
        (self.w_cost * cab_option.fare) - \
        (self.w_time * cab_option.eta)
    return u
def run_agent(self, percept):
    pickup = percept['current_location']
    dropoff = percept['destination']
    cab_types = ['micro', 'mini', 'sedan', 'prime', 'shared']
    # Step 1: Query available nearby cabs for each type
    available_options = []
    for c_type in cab_types:
        cabs = query_available_cabs(c_type, pickup)
        if cabs:
            fare = calculate_fare(c_type, pickup, dropoff)
            eta = estimate_pickup_time(cabs, pickup)
            comfort = get_comfort_rating(c_type)
```

```
        available_options.append(CabOption(c_type, cabs, fare, eta, comfort))
if not available_options:
    return "No cabs currently available"
# Step 2: Rank options using the utility function
best_option = None
max_utility = -float('inf')
for option in available_options:
    u = self.utility(option)
    if u > max_utility:
        max_utility = u
        best_option = option
# Step 3: Execute Action
booking_confirmation = book_cab(
    cab_type=best_option.cab_type,
    pickup=pickup,
    destination=dropoff,
    selected_driver=best_option.nearest_driver()
)
return booking_confirmation
```

5. Least-Cost Path using Uniform Cost Search (UCS)

Problem Graph

Nodes: S (Start), A, B, C, D, G (Goal)

Weighted Directed Edges:

- S → A: 1
- S → G: 12
- A → B: 3
- A → C: 1
- B → D: 3
- C → D: 1
- C → G: 2
- D → G: 3

Uniform Cost Search Algorithm Execution

Uniform Cost Search (UCS) expands nodes in order of non-decreasing path cost g(n). It uses a priority queue, where the node with the lowest cumulative path cost is selected first.

Step 1: Initialization

- Frontier: [S, g = 0]
- Explored Set: ø

Step 2: Expand S

Pop S with g = 0.

Generate successors:

- S → A: 0 + 1 = 1
- S → G: 0 + 12 = 12

Frontier:

- S → A, g = 1
- S → G, g = 12

Step 3: Expand A

Pop A with g = 1.

Generate successors:

- S → A → B: 1 + 3 = 4
- S → A → C: 1 + 1 = 2

Frontier:

- S → A → C, g = 2
- S → A → B, g = 4
- S → G, g = 12

Step 4: Expand C

Pop C with g = 2.

Generate successors:

- S → A → C → D: 2 + 1 = 3
- S → A → C → G: 2 + 2 = 4

Frontier:

- S → A → C → D, g = 3
- S → A → B, g = 4
- S → A → C → G, g = 4
- S → G, g = 12

Step 5: Expand D

Pop D with g = 3.

Generate successor:

- S → A → C → D → G: 3 + 3 = 6

Frontier:

- S → A → B, g = 4
- S → A → C → G, g = 4

- S → A → C → D → G, g = 6
- S → G, g = 12

Step 6: Expand B

There is a tie between B and G, both having g = 4. Select B first.

Pop B with g = 4.

Generate successor:

- S → A → B → D: 4 + 3 = 7

Since D has already been reached with a lower cost of 3, this path is discarded.

Frontier:

- S → A → C → G, g = 4
- S → A → C → D → G, g = 6
- S → G, g = 12

Step 7: Expand G

Pop G with g = 4.

Since G is the goal node, UCS terminates.

Summary of Results

Optimal Least-Cost Path:

S → A → C → G

Total Path Cost:

1 + 1 + 2 = 4

Therefore,

Optimal Path = S → A → C → G

Minimum Cost = 4

Conclusion

Using Uniform Cost Search, the optimal least-cost path from the starting warehouse S to the destination warehouse G is S → A → C → G, with a minimum total transportation cost of 4.